The Laundry List

THE OBJECTION

Anthropic built an executor.

THE UNKNOWNS

Every unknown lives in one of four boxes.

	WHAT YOU KNOW	WHAT YOU DON'T
YOU'RE AWARE	**Known knowns** what's already in your prompt	**Known unknowns** the gaps you can name
YOU'RE NOT	**Unknown knowns** obvious to you, never written down	**Unknown unknowns** what you never thought to ask

Your prompt only fills the first box. **The wargame drags the other three into the light.**

THE MOVE

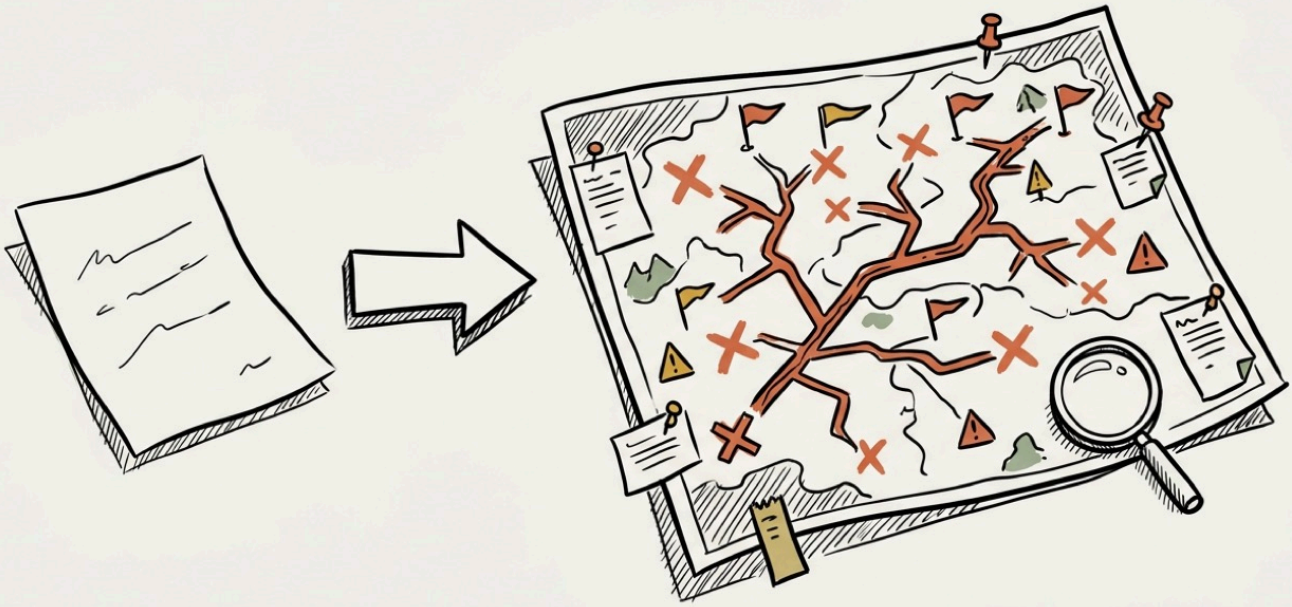
Make it fight the build on paper.

THE STANDARD

Wargamed means it survives contact.

THE PAYOFF

From outline to battle plan.



01 Build the Website

CODE XHIGH

→ **You get** the site build fought on paper, every move with its expected observation and failure branch

```
WARGAME ORDER. You are not executing this mission, you are wargaming it. A
cheaper executor (Sonnet) runs the brief below later. Your job is the route
it will follow.

Recon first, read-only: the reference site and anything the build must match.

Then fight the mission on paper, move by move, and write it to wargames/01-
website.md:

- every move states its expected observation, exactly what you should see if
it worked
- every move carries its most likely failure, the cause it signals, and the
counter-move
- every fork gets a trigger, if you observe X, take route B
- assumptions recon could not settle get marked RECON NEEDED with the exact
check that settles it
- end with abort conditions, and the verification runs the executor must
perform with what pass looks like for each

Write it so the executor can run the brief end to end without asking a single
```

question.

=== THE MISSION BRIEF (the executor's orders, not yours) ===

I'm rebuilding the marketing site for **{{BUSINESS}}** because the current one **{{PROBLEM, e.g. looks dated and doesn't convert}}**. Visitors are **{{AUDIENCE}}** and the one action I want from them is **{{CTA}}**.

Build a complete static site in ./site. Plain HTML, CSS, and JS. No frameworks, no build step, opening index.html in a browser shows the finished site. Sections: **{{LIST THEM}}**. Match this reference for tone and palette: **{{URL OR DESCRIPTION}}**.

Mobile first. No horizontal scroll at 375px. Semantic landmarks, labeled form inputs, alt text on every image.

When you believe you are done, verify before reporting. Open each page, exercise every link, every form validation path, and every interactive element, and fix what fails. Audit each claim in your final summary against something you actually ran or read in this session.

Do the simplest thing that works well. No features, no abstractions, nothing beyond this list.

02 Write the Copy

COPY HIGH

→ **You get** the copy mission wargamed, section order, voice risks, and the skeptic pass pre-planned

```
WARGAME ORDER. You are not executing this mission, you are wargaming it. A
cheaper executor (a mid-tier model) runs the brief below later. Your job is
the route it will follow.

Recon first, read-only: the current page and every voice sample I gave you.

Then fight the mission on paper, move by move, and write it to wargames/02-
copy.md:

- every move states its expected observation, exactly what you should see if
it worked
- every move carries its most likely failure, the cause it signals, and the
counter-move
- every fork gets a trigger, if you observe X, take route B
- assumptions recon could not settle get marked RECON NEEDED with the exact
check that settles it
- end with abort conditions, and the verification runs the executor must
perform with what pass looks like for each

Write it so the executor can run the brief end to end without asking a single
```

question.

=== THE MISSION BRIEF (the executor's orders, not yours) ===

Write the full copy for **{{PAGE, e.g. the homepage from task 01}}**. The reader is **{{ICP}}**. They arrive **{{STATE OF MIND, e.g. skeptical, burned by two agencies}}**. This page has one job, moving them to **{{CTA}}**.

Voice: **{{THREE ADJECTIVES}}**, in the spirit of **{{WRITER OR BRAND YOU ADMIRE}}**.

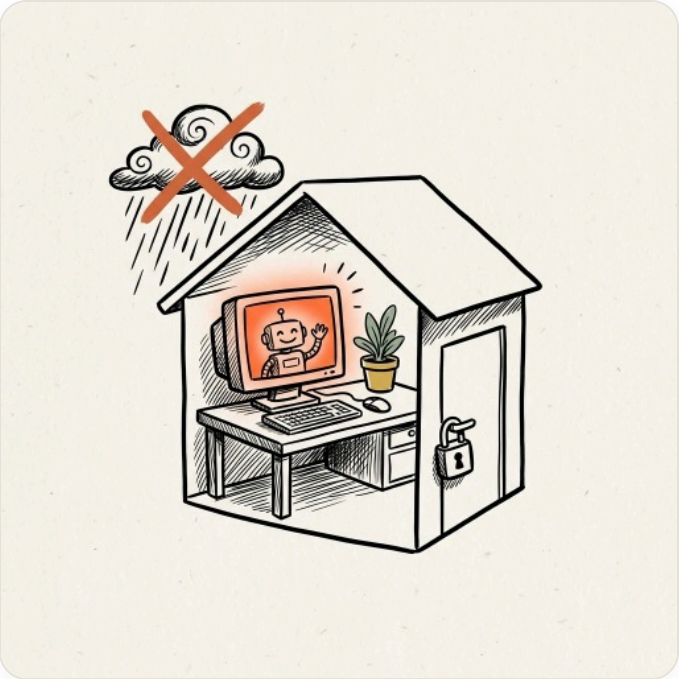
Draft every section. Headline, subhead, three benefit blocks, proof section, FAQ, closing CTA. Write two alternates for the headline and the CTA only, nothing else gets variants.

Rules. Lead with the outcome the reader gets. Plain words over clever ones. No hype adjectives. Sentences a 7th grader can read. Being readable and being concise are different things, and readability wins.

Before you finish, reread the entire page as a skeptical **{{ICP}}** who has seen ten pages like this today, and cut every line that does not move them toward **{{CTA}}**.

03 Set Up Local AI

LOCAL AI HIGH



→ **You get** your exact machine's local stack wargamed, runtime, models, quants, fallbacks, and the speed checks

```
WARGAME ORDER. You are not executing this mission, you are wargaming it. A
cheaper executor (Claude Code on a cheaper model) runs the brief below later.
Your job is the route it will follow.

Recon first, read-only: the machine specs I gave you, and the current
releases of every tool you plan to recommend.

Then fight the mission on paper, move by move, and write it to wargames/03-
localai.md:

- every move states its expected observation, exactly what you should see if
it worked
- every move carries its most likely failure, the cause it signals, and the
counter-move
- every fork gets a trigger, if you observe X, take route B
- assumptions recon could not settle get marked RECON NEEDED with the exact
check that settles it
- end with abort conditions, and the verification runs the executor must
perform with what pass looks like for each
```

Write it so the executor can run the brief end to end without asking a single question.

=== THE MISSION BRIEF (the executor's orders, not yours) ===

I want a fully local, open source AI setup on this machine, private by default, nothing leaves the box. My hardware: **{{OS AND VERSION}}**, **{{CHIP, e.g. M2 Pro}}**, **{{RAM}}**, **{{GPU / VRAM IF ANY}}**, **{{FREE DISK}}**. I'll use it for: **{{USE CASES, e.g. private doc chat, coding help, first drafts}}**. My patience for tinkering: **{{LOW / MEDIUM / HIGH}}**.

Set up the stack that fits THIS machine, not a generic tutorial. Pick the runtime and justify it against my patience level. Pick the exact models with the exact quantizations that fit my memory, one daily driver, one small fast fallback, plus an embedding model if my use cases need document chat. Configure context length and GPU offload for my hardware.

Verify the whole thing end to end. A test prompt runs on each model with tokens per second measured, and each of my use cases gets exercised once. Confirm nothing phones home, the setup works with wifi off.

Everything must be free and open source. If my hardware cannot run a good daily driver, say so plainly and name the smallest upgrade that changes that.

04 The Tax Strategy Review

FINANCE XHIGH

→ **You get** the tax memo route with every unverifiable number flagged RECON NEEDED before your accountant sees it

```
WARGAME ORDER. You are not executing this mission, you are wargaming it. A
cheaper executor (Opus) runs the brief below later. Your job is the route it
will follow.

Recon first, read-only: every statement and category I attached.

Then fight the mission on paper, move by move, and write it to wargames/04-
tax.md:

- every move states its expected observation, exactly what you should see if
it worked
- every move carries its most likely failure, the cause it signals, and the
counter-move
- every fork gets a trigger, if you observe X, take route B
- assumptions recon could not settle get marked RECON NEEDED with the exact
check that settles it
- end with abort conditions, and the verification runs the executor must
perform with what pass looks like for each
```

Write it so the executor can run the brief end to end without asking a single question.

=== THE MISSION BRIEF (the executor's orders, not yours) ===

I run **{{ENTITY TYPE, e.g. a Canadian corporation}}** in **{{JURISDICTION}}** doing roughly **{{REVENUE}}** per year, with **{{STRUCTURE NOTES, e.g. one owner-operator, no payroll, home office}}**. Raw materials: **{{STATEMENTS / EXPENSE CATEGORIES / ANYTHING YOU HAVE}}**.

Act as my accountant's analyst, not my accountant. Produce a tax strategy memo I will hand to a professional for review.

The memo covers. My current posture in plain language. Every deduction, deferral, and structure opportunity I appear to be missing, each with the rule it relies on, an estimated impact range, and the documentation I would need. Aggressive positions flagged separately from safe moves, clearly labeled. And the list of questions a good accountant would ask me next.

Audit every number in the memo against the materials I gave you. Never estimate silently. Anything you cannot verify from my materials gets marked unverified. This memo informs a conversation, it does not replace one.

05 Refine the High-Ticket Offer

OFFER DESIGN XHIGH

→ **You get** the offer rebuild wargamed, buyer counterattacks and their patches already fought

```
WARGAME ORDER. You are not executing this mission, you are wargaming it. A
cheaper executor (a mid-tier model) runs the brief below later. Your job is
the route it will follow.

Recon first, read-only: the current pitch and the objection list.

Then fight the mission on paper, move by move, and write it to wargames/05-
offer.md:

- every move states its expected observation, exactly what you should see if
it worked
- every move carries its most likely failure, the cause it signals, and the
counter-move
- every fork gets a trigger, if you observe X, take route B
- assumptions recon could not settle get marked RECON NEEDED with the exact
check that settles it
- end with abort conditions, and the verification runs the executor must
perform with what pass looks like for each

Write it so the executor can run the brief end to end without asking a single
```

question.

=== THE MISSION BRIEF (the executor's orders, not yours) ===

I sell **{{PROGRAM, e.g. a high-ticket AI implementation program}}** at **{{PRICE}}** to **{{ICP, e.g. entrepreneurs doing 500K to 5M}}**. Close rate is roughly **{{X}}**%. The objections I hear most: **{{LIST THEM HONESTLY}}**. Current pitch or sales page: **{{PASTE}}**.

Rebuild the offer, not the copy.

Deliver. The one painful, expensive problem this offer should anchor on, in the buyer's own words. The promise, restated so the price feels obvious rather than defended. What to add, what to cut, and what to guarantee, each with the reason. Risk reversal options, ranked by how much backbone they require. And the three hardest questions a skeptical **{{ICP}}** would ask, with honest answers, not deflections.

Then pressure-test it. Argue against your own offer as a buyer who almost bought and walked away, and patch whatever that argument exposes. Recommendation first, reasoning after.

06 Upgrade the Chatbot From Real Conversations

AI SYSTEMS HIGH

→ **You get** the chatbot upgrade route, failure patterns quoted, rewrite moves with expected outcomes

```
WARGAME ORDER. You are not executing this mission, you are wargaming it. A
cheaper executor (Sonnet) runs the brief below later. Your job is the route
it will follow.

Recon first, read-only: all {{N}} transcripts end to end.

Then fight the mission on paper, move by move, and write it to wargames/06-
chatbot.md:

- every move states its expected observation, exactly what you should see if
it worked
- every move carries its most likely failure, the cause it signals, and the
counter-move
- every fork gets a trigger, if you observe X, take route B
- assumptions recon could not settle get marked RECON NEEDED with the exact
check that settles it
- end with abort conditions, and the verification runs the executor must
perform with what pass looks like for each
```

Write it so the executor can run the brief end to end without asking a single question.

=== THE MISSION BRIEF (the executor's orders, not yours) ===

Attached are **{{N}}** real conversations from my **{{CHATBOT PURPOSE, e.g. lead qualification}}** bot: **{{FILES OR PASTE}}**. Its current system prompt: **{{PASTE}}**.

Find where it actually fails. Wrong answers, missed handoffs, tone breaks, users repeating themselves, conversations that dead-end. Group the failures into named patterns, and support every pattern with at least one direct quote from the transcripts. If you cannot quote it, it does not exist.

Then rewrite the system prompt to fix the top patterns. Deliver the new prompt, plus a change log, each change paired with the failure pattern it prevents. Keep the new prompt as short as it can be while fixing what you found.

WATCH OUT

Do not ask the model to explain its thinking or reproduce its reasoning in the output. On Fable that can trigger the reasoning-extraction safeguard and silently route your session to Opus 4.8. Ask for artifacts, findings, quotes, and rewrites, never for the thinking itself.

07 Hunt the Bugs

CODE XHIGH

→ **You get** a bug-hunt wargame, candidate bugs ranked with the exact verification run for each

```
WARGAME ORDER. You are not executing this mission, you are wargaming it. A
cheaper executor (Claude Code on a cheaper model) runs the brief below later.
Your job is the route it will follow.

Recon first, read-only: the whole repo, trace the core flows.

Then fight the mission on paper, move by move, and write it to wargames/07-
bugs.md:

- every move states its expected observation, exactly what you should see if
it worked
- every move carries its most likely failure, the cause it signals, and the
counter-move
- every fork gets a trigger, if you observe X, take route B
- assumptions recon could not settle get marked RECON NEEDED with the exact
check that settles it
- end with abort conditions, and the verification runs the executor must
perform with what pass looks like for each

Write it so the executor can run the brief end to end without asking a single
```

question.

=== THE MISSION BRIEF (the executor's orders, not yours) ===

Here is my repo: {{PATH}}. Before touching anything, read the README and trace the core flow end to end so you understand what the system is supposed to do.

Then hunt real bugs. Logic errors, unhandled edge cases, race conditions, data-loss paths, security holes at the boundaries. For every finding, cite the file and line, describe the failure scenario in one sentence, rate the severity, and prove it is real with a failing test, a reproduction command, or a concrete trace through the code. If you cannot point to evidence, it does not go in the report.

No style nits. No refactors. No hypothetical hardening for scenarios that cannot happen.

Fix only the top {{N}} findings, each with the smallest change that works, and run the test suite before and after so the report shows both results.

08 Build the Financial Model

FINANCE HIGH

→ **You get** the model build wargamed, formulas, levers, and sensitivity checks pre-fought

```
WARGAME ORDER. You are not executing this mission, you are wargaming it. A
cheaper executor (a mid-tier model) runs the brief below later. Your job is
the route it will follow.

Recon first, read-only: the revenue and cost inputs I gave you.

Then fight the mission on paper, move by move, and write it to wargames/08-
model.md:

- every move states its expected observation, exactly what you should see if
it worked
- every move carries its most likely failure, the cause it signals, and the
counter-move
- every fork gets a trigger, if you observe X, take route B
- assumptions recon could not settle get marked RECON NEEDED with the exact
check that settles it
- end with abort conditions, and the verification runs the executor must
perform with what pass looks like for each

Write it so the executor can run the brief end to end without asking a single
```

question.

=== THE MISSION BRIEF (the executor's orders, not yours) ===

Build a 12-month financial model for **{{BUSINESS}}** as a spreadsheet, saved as **{{NAME}}**.xlsx.

One inputs sheet holding every assumption: **{{REVENUE STREAMS}}**, **{{COST LINES}}**, **{{ASSUMPTIONS, e.g. churn, close rate, price changes}}**. Each assumption lives in one labeled cell. Nothing gets buried inside a formula.

Model monthly cash across base, bear, and bull scenarios, driven by the three levers I am most likely to change: **{{LEVERS}}**. Add a summary sheet a non-finance person can read in sixty seconds.

Use real formulas, not hardcoded values. Before reporting, sanity-check the model, move each lever and confirm the outputs shift the way reality would. Close with the three assumptions the model is most sensitive to, so I know what to watch.

09 Tear Down the Competition

RESEARCH HIGH

→ **You get** the teardown route, sources to pull, conflicts expected, and the gap-map criteria set

```
WARGAME ORDER. You are not executing this mission, you are wargaming it. A
cheaper executor (Opus) runs the brief below later. Your job is the route it
will follow.

Recon first, read-only: each competitor property you can reach.

Then fight the mission on paper, move by move, and write it to wargames/09-
competitors.md:

- every move states its expected observation, exactly what you should see if
it worked
- every move carries its most likely failure, the cause it signals, and the
counter-move
- every fork gets a trigger, if you observe X, take route B
- assumptions recon could not settle get marked RECON NEEDED with the exact
check that settles it
- end with abort conditions, and the verification runs the executor must
perform with what pass looks like for each

Write it so the executor can run the brief end to end without asking a single
```

question.

=== THE MISSION BRIEF (the executor's orders, not yours) ===

I'm positioning **{{BUSINESS}}** against these competitors: **{{3 TO 5 NAMES OR URLS}}**.

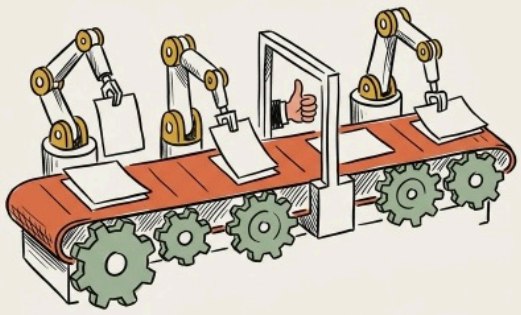
For each competitor, establish four things. What they actually sell, not what their homepage claims. Price points you can verify. The promise their positioning makes. And their visible weakness in their own customers' words, from reviews, comments, and complaints.

Cite a source for every claim. Anything you cannot verify gets marked unverified rather than smoothed over. Where sources conflict, say so instead of averaging.

Then the deliverable that matters, the gap map. What does **{{ICP}}** want that nobody on this list credibly owns, and what is the one positioning move I could defend against all of them? End with that recommendation and the evidence trail behind it.

10 Map the Automation

OPERATIONS HIGH



→ **You get** the automation blueprint wargamed, checkpoints, what breaks first, and abort lines per phase

```
WARGAME ORDER. You are not executing this mission, you are wargaming it. A
cheaper executor (Claude Code on a cheaper model) runs the brief below later.
Your job is the route it will follow.

Recon first, read-only: the process description and every tool it touches.

Then fight the mission on paper, move by move, and write it to wargames/10-
automation.md:

- every move states its expected observation, exactly what you should see if
it worked
- every move carries its most likely failure, the cause it signals, and the
counter-move
- every fork gets a trigger, if you observe X, take route B
- assumptions recon could not settle get marked RECON NEEDED with the exact
check that settles it
- end with abort conditions, and the verification runs the executor must
perform with what pass looks like for each

Write it so the executor can run the brief end to end without asking a single
```

question.

=== THE MISSION BRIEF (the executor's orders, not yours) ===

Here is a process I currently run by hand: **{{DESCRIBE IT STEP BY STEP, WITH THE TOOLS EACH STEP TOUCHES}}**.

Map it into an automation blueprint. Classify every step as automate fully, automate with a human checkpoint, or keep human, with the reason. For each automated step, name the tool or script that does it. For the whole pipeline, identify what breaks first and the guardrail for it.

Sequence the build starting with the step that saves the most time per week. Each phase of the build gets its own acceptance check, something I can run to confirm that phase works before starting the next.

Delegate independent steps to subagents and keep working while they run. The final output is a blueprint I can hand to Claude Code and say, build phase one.

THE MACHINE

RUN IT Don't Run Them One by One. Run the List.

The move. Save each filled-in mission as its own file. One **/goal** contract makes Fable draft a wargame for all ten, breadth first. Then **/loop** takes over, grading every draft against the eight-point definition below, red-teaming the weakest, patching what breaks, until each one survives an honest attempt to kill it.

Step 1 · The folder. The kit ships this folder ready. cd in, fill the placeholders in tasks/, and remember, an unfilled placeholder means that mission is blocked by definition.

```
cd {{PATH TO YOUR PROJECTS}}/fable-last-week

fable-last-week/
  tasks/       01-website.md ... 10-automation.md   (the ten missions)
  wargames/    starts empty, Fable fills it
  SUCCESS.md   the eight-point definition of properly wargamed
  LEDGER.md    every grade and every patch, logged as it goes
```

Step 2 · The standard. Save this as SUCCESS.md. It is the definition of done the loop grades against, be stricter here and everything downstream gets better.

```
# SUCCESS.md · the definition of properly wargamed

A wargame passes only when ALL eight hold.

1. Every move states its expected observation, exactly what you should see
if the move worked.

2. Every move carries its most likely failure, the cause that failure
signals, and the counter-move.

3. Every fork has a trigger. If you observe X, take route B. No judgment
calls left to the executor.

4. Every assumption recon could not settle is marked RECON NEEDED with the
exact check that settles it.
```

```
5. Abort conditions exist, the moments to stop and flag rather than
improvise.

6. Verification is spelled out, which runs the executor performs, when, and
what pass looks like for each.

7. It has survived a red-team pass. The doc records the attack that failed
against it, and the patch born from the attack that did not.

8. It is executable blind. A mid-tier model could run the mission end to end
without asking a single question.
```

Step 3 · The contract. This is the exact /goal prompt. Target, criteria, proof, blockers, turn cap, stop condition, autonomy. Paste it and walk away.

```
/goal Every mission file in ./tasks has a first-draft wargame in ./wargames,
logged in LEDGER.md with a self-grade against SUCCESS.md.

The contract.

1. Each file in ./tasks is a mission. The mission text is the executor's
definition of done. You do not execute any mission this week, you wargame
it.

2. Recon is read-only. Read anything you need, run nothing that changes
state.

3. For each mission, write wargames/.md, the route move by move: expected
observation per move, most likely failure with the cause it signals and the
counter-move, triggers that reroute, RECON NEEDED marks with the exact
settling check, abort conditions, and the executor's verification runs with
what pass looks like.

4. Draft all ten before polishing any. Breadth first, the refinement loop
owns depth.

5. After each draft, append a LEDGER.md entry: the mission, the draft's
location, and an honest point-by-point self-grade against all eight points
of SUCCESS.md.

6. A mission with an unfilled {{PLACEHOLDER}} is BLOCKED. Write what you
```

```
need in LEDGER.md and move on. Never invent the missing input.

7. You are operating autonomously. I am not watching in real time. Before
you end a turn, check your last paragraph. If it is a plan, a question, or a
promise about work not yet done, do that work now instead.

8. Stop when all ten missions are DRAFTED or BLOCKED in LEDGER.md.
```

Step 4 · The refinement loop. This is where drafts become wargames. Every cycle it grades all ten, takes the weakest, tries to break it, and patches what breaks. It defines its own finish line and shuts itself off when every mission holds.

```
/loop 20m Loop through every first draft in ./wargames until each one is
properly wargamed to the best of your ability. Properly wargamed means all
eight points of SUCCESS.md hold, no exceptions.

Each cycle: grade every draft point by point against SUCCESS.md and log the
grades in LEDGER.md. Take the weakest draft and red-team it, play the
executor following it blind and attack the route, find the move where it
breaks. Patch the break, add the branch that catches it next time, upgrade
vague moves with expected observations, convert every unstated assumption to
a RECON NEEDED mark with its settling check. Re-grade and log what changed.

A wargame is DONE when it passes all eight points AND one honest attempt to
break it fails. Do not soften the grading to finish faster, a draft that
passes on paper but dies at first contact is a failure of this loop.

When every wargame is DONE or BLOCKED, or two consecutive cycles improve
nothing, post the final ledger and stop the loop.
```

BUDGET THE CAP

You get Fable at 50% of your weekly limits through July 7, and wargaming is judgment-dense but token-light, no edit loops, no test runs, so the cap goes far. Run it at effort xhigh, this is exactly the work the deep thinking is for. If the cap gets tight, drop the refinement loop to high and keep the drafting pass at xhigh.

Ten tasks. Five days. **Make the smartest model you'll ever rent do the thinking while it's still on salary.**